Parte A. Análisis y diseño

A.1. Selección de colecciones

La selección de estructuras de datos se realizó teniendo en cuenta las operaciones que necesita el sistema de gestión de paquetes de Quindío Express, la frecuencia con la que se ejecutan, el orden requerido y la cantidad de información que se espera administrar.

No existe una única estructura que sea la mejor para todas las necesidades. Por esta razón, el sistema combina distintas colecciones del Java Collections Framework, de manera que cada una se encargue de la operación para la cual resulta más apropiada.

Necesidad	Estructura elegida	Operación principal	Complejidad esperada	Alternativa descartada	Justificación
1. Mantener paquetes pendientes en el orden de registro	ArrayDeque	Insertar al final y consultar o retirar el primero	Inserción y retiro por extremos: $O(1)$ amortizado	ArrayList	Permite manejar los paquetes como una cola FIFO (First In, First Out), conservando el orden de llegada. En un ArrayList, retirar el primer elemento exige desplazar los restantes, operación $O(n)$.
2. Consultar rápidamente un paquete mediante su código	HashMap	Buscar por código	$O(1)$ promedio; $O(n)$ en el peor caso teórico	ArrayList	El código funciona como clave y permite acceder directamente al objeto asociado sin recorrer todos los paquetes en condiciones normales.
3. Mantener	HashSet	Agregar y	$O(1)$	ArrayList	Evita

municipios destino sin duplicados		verificar existencia	promedio		duplicados automáticamente y permite comprobar la existencia de un municipio sin recorrer una lista completa en el caso promedio.
4. Mantener municipios ordenados alfabéticamente y sin repetidos	TreeSet	Insertar, buscar y recorrer en orden	$O(\log m)$ para inserción y búsqueda; recorrido completo $O(m)$	HashSet	Mantiene los elementos ordenados según su criterio de comparación y evita duplicados. HashSet no garantiza un orden alfabético.
5. Atender paquetes por mayor prioridad	PriorityQueue	Consultar y retirar el paquete prioritario	Consultar el primero $O(1)$; insertar y retirar el primero $O(\log n)$	ArrayDeque	Permite que el paquete con mayor prioridad se atienda antes que los de menor prioridad. Una cola FIFO solo considera el orden de llegada.
6. Mantener el historial de paquetes entregados en orden de entrega	ListaSimple propia	Agregar una entrega al final y recorrer el historial	Agregar al final $O(1)$; recorrer $O(n)$	ArrayList	Se utiliza la lista simplemente enlazada, cumpliendo el requisito de implementar una estructura propia y utilizarla para almacenar el historial.
7. Asociar cada código con su objeto	HashMap	Asociar código y objeto, y	$O(1)$ promedio para inserción y	TreeMap	HashMap es apropiado

Paquete		consultar por código	búsqueda		porque no se necesita ordenar los códigos para consultarlos. TreeMap mantendría las claves ordenadas, pero sus operaciones principales tendrían un costo de $O(\log n)$.

A.2. Tiempo frente a memoria

Quindío Express administra inicialmente 30 paquetes, pero espera crecer hasta aproximadamente 500.000. La operación más frecuente será consultar un paquete a partir de su código.

Se comparan ArrayList<Paquete> y HashMap<String, Paquete>.

Criterio	**ArrayList**	**HashMap**
Forma de almacenamiento	Secuencia de referencias a paquetes	Entradas que asocian una clave con un objeto
Búsqueda por código	$O(n)$, porque debe examinar los paquetes hasta encontrar el código	$O(1)$ promedio; $O(n)$ en el peor caso teórico
Tiempo con 30 paquetes	Puede ser suficiente por el tamaño reducido	También es eficiente, aunque puede no apreciarse una diferencia significativa
Tiempo con 500.000 paquetes	Una búsqueda puede revisar una parte considerable de los elementos; en el peor caso revisa los 500.000	En condiciones normales, la búsqueda depende de calcular el hash y localizar la entrada, no de recorrer todos los paquetes
Memoria adicional	Relativamente baja: arreglo de referencias y capacidad reservada	Mayor: tabla hash, entradas, claves y estructura interna adicional
Ventaja principal	Sencillez, recorrido secuencial eficiente y poco sobrecosto estructural	Acceso directo por código y asociación explícita entre clave y objeto

Desventaja principal	Las búsquedas por código se vuelven más costosas a medida que aumenta la cantidad de paquetes	Consume más memoria y depende de una distribución adecuada de los valores hash
Uso recomendado	Listados, recorridos y situaciones donde predomina el acceso secuencial	Consultas frecuentes por código único

Para un ArrayList, la búsqueda por código requiere comparar el código buscado con el de cada paquete hasta encontrar una coincidencia.

En el peor caso: $T(n) = O(n)$

Si hay 500.000 paquetes, una búsqueda puede llegar a comparar el código con los 500.000 elementos. Si la posición buscada se distribuye uniformemente y el código existe, se realizarían alrededor de 250.000 comparaciones en promedio como aproximación simplificada.

Para un HashMap, la búsqueda utiliza el código como clave. El mapa calcula su valor hash y localiza la entrada correspondiente.

Con una distribución adecuada, el costo esperado es:$T(n) = O(1)promedio$

No significa que todas las consultas tarden exactamente lo mismo ni que el peor caso sea siempre constante. La implementación de la tabla hash, las colisiones y la calidad de los códigos influyen en su rendimiento.

Análisis de memoria

ArrayList requiere un arreglo interno de referencias a objetos Paquete. HashMap necesita, además, mantener las claves, las entradas y la estructura de dispersión. Por eso, la tabla hash generalmente tiene un mayor consumo de memoria por asociación almacenada.

Sin embargo, ambos pueden referenciar los mismos objetos Paquete; usar un HashMap no implica necesariamente duplicar todos los objetos completos. El gasto adicional depende de las referencias, claves y entradas que necesite el mapa.

Decisión final

Se selecciona HashMap<String, Paquete> como índice de consulta por código, porque la búsqueda por identificador es la operación dominante y el volumen esperado puede alcanzar 500.000 paquetes. Aunque utiliza más memoria que un ArrayList, el costo adicional se justifica por la reducción del tiempo esperado de búsqueda.

Esto no significa que deba eliminarse el ArrayList utilizado para registrar o recorrer los paquetes. Ambas estructuras pueden coexistir: la lista sirve para conservar un registro ordenado de paquetes y el mapa funciona como índice de consulta rápida.

Esta decisión se refleja en el diseño de CentroDistribucion, que mantiene un registro de paquetes y un índice por código.

Parte C. Análisis algorítmico y pruebas

C.1. Pila de llamadas

En QuindíoExpress se implementan dos operaciones recursivas principales y un algoritmo de divide y vencerás:

- pesoTotalPorDestino: calcula el peso acumulado de los paquetes que tienen un destino específico.
- contarPrioridadMinima: cuenta los paquetes cuya prioridad es igual o superior a un valor mínimo.
- masPesado: divide el conjunto de paquetes en dos partes, encuentra el más pesado en cada una y compara ambos resultados.

Pruebas de escritorio y ejecuciones de pila: Consolidado completo disponible en este enlace a Google Sheets.
https://docs.google.com/spreadsheets/d/1RnjfRd2SwLFOrFjXGrbU8vA_6t93CgyRsNyEv14OoZc/edit?usp=sharing

Parte D. Uso justificado de inteligencia artificial

D.1. Bitácora

Por cada consulta relevante realizada a una herramienta de inteligencia artificial registren:

Herramienta	**¿Qué preguntaron?**	**¿Por qué la consultaron?**	**¿Cómo verificaron la respuesta?**	**¿Qué modificaron?**
ChatGPT	¿Qué estructura de datos conviene utilizar para consultar paquetes por código cuando el sistema puede crecer hasta 500.000 registros?	Se necesitaba seleccionar una estructura que permitiera realizar consultas frecuentes sin recorrer todos los paquetes.	Se comparó la complejidad de ArrayList y HashMap, y se revisó que el código permitiera consultar los paquetes mediante su identificador.	Se utilizó un HashMap<String, Paquete> como índice por código, manteniendo una lista para el registro general.

ChatGPT	¿Cuál es la diferencia entre Comparable y Comparator, y cómo se pueden utilizar para ordenar paquetes por diferentes criterios?	El sistema necesitaba un orden natural por código y criterios alternativos por prioridad, peso y tiempo estimado.	Se verificaron las implementaciones de compareTo y de los comparadores, y se ejecutaron pruebas de ordenamiento.	Se implementó Comparable<Paquete> para el orden natural y se definieron comparadores independientes para los criterios alternativos.
ChatGPT	¿Cómo implementar una lista simplemente enlazada genérica que permita recorrer sus elementos con for-each?	El enunciado exigía implementar una lista propia y utilizarla para almacenar el historial de entregas.	Se comprobó que la lista tuviera nodos propios, que implementara Iterable<T> y que el iterador cumpliera el contrato de Iterator<T>. También se probaron listas vacías, eliminación de elementos e inserciones posteriores.	Se utilizaron una clase Nodo<T>, una clase ListaSimple<T> y una clase IteradorLista<T>, sin reemplazar internamente la estructura por una colección del framework.
ChatGPT	¿Cómo aplicar recursividad para sumar el peso de los paquetes por municipio y contar los que cumplen una prioridad mínima?	Se necesitaba resolver operaciones de análisis mediante recursividad, sin sustituirla por ciclos.	Se realizaron pruebas manuales con pocos paquetes, identificando los casos base y comprobando los resultados de las sumas y los conteos.	Se implementaron métodos recursivos que procesan un paquete por llamada y utilizan el final de la lista como condición de parada.

ChatGPT	¿Cómo implementar una búsqueda binaria para localizar un paquete mediante su código y cómo comprobar que la lista cumple la precondición?	Se necesitaba demostrar el uso correcto de búsqueda binaria y evitar resultados incorrectos al trabajar con datos no ordenados.	Se verificó que la lista estuviera ordenada por código y se probó la búsqueda tanto con un código existente como con uno inexistente.	Se implementó una verificación del orden y se ejecutó la búsqueda sobre una copia ordenada del registro.
ChatGPT	¿Cómo utilizar divide y vencerás para encontrar el paquete de mayor peso?	Se requería un algoritmo que dividiera el problema en subproblemas y combinara los resultados, en lugar de limitarse a una búsqueda convencional.	Se examinó la división de intervalos, los casos base y la comparación entre los resultados de las dos mitades.	Se implementó masPesado de forma recursiva, comparando los máximos de cada subintervalo hasta obtener el paquete más pesado.

D.2. Una respuesta cuestionable de IA

Caso seleccionado: sustituir la lista simplemente enlazada propia por LinkedList.

Respuesta original	Problema detectado	Análisis	Solución final
Para almacenar el historial de entregas, utiliza LinkedList<Paquete>. Esta colección ya implementa una lista enlazada, permite agregar y eliminar elementos y evita que tengas que implementar nodos, enlaces e iteradores manualmente	La recomendación utiliza una colección existente del Java Collections Framework en lugar de desarrollar la lista propia exigida en el enunciado.	Aunque LinkedList es una estructura válida para determinadas operaciones, la solución incumple una restricción explícita: implementar una lista simplemente enlazada propia sin utilizar internamente ArrayList, LinkedList u otra colección del framework. Por lo tanto, el problema no es	Se implementó ListaSimple<T> mediante nodos enlazados, con referencias al primer y último nodo y un contador de elementos. La clase implementa Iterable<T> y proporciona un Iterator<T> propio. El historial de entregas

		necesariamente la eficiencia de LinkedList, sino que no demuestra la implementación de nodos, enlaces, genéricos e iteradores propios.	utiliza esta lista para conservar el orden en que se registran las entregas.